

From Algorithm to Program

TC1037 · Algorithmic Thinking · Lecture 02

Dr. Leopoldo Rodríguez Salazar

Friday 25 September 2026

Tecnológico de Monterrey · Campus Querétaro · Group 510

A famous algorithm you already use badly

You are looking for an apartment.

Querétaro, September. Every one you see is gone by tomorrow.
You can visit at most **ten**. You must say yes on the spot or lose it.

When do you stop looking?

Algorithms to Live By, Christian & Griffiths (2016), ch. 1 — *Optimal Stopping*

The 37% rule

1. Take the first **37%** of the options and *only look*. Say no to every one of them — but remember the best.
2. After that, take the **first one that beats everything you saw**.
3. If none does, you get the last one.

$37\% = 1/e$. Ten apartments: look at 3 (or 4), then leap.

Strategy	P(best)
Take the first one, no looking	10%
Look at 1, then leap	28%
Look at 3–4, then leap	40%
Look at 5, then leap	37%
Look at 8, then leap	19%
See all ten, pick the best	impossible
P(choosing the best) with 10 options. With many options the best cut-off tends to 37%, and so does the success rate.	

It is the best possible strategy — and it fails 63% of the time.

Your turn — 3 minutes, on paper

Write the 37% rule as an algorithm, in Tuesday's notation.

INPUT: OUTPUT: numbered steps.

Ten apartments, seen one at a time; you cannot go back.

- What exactly do you *remember* during the looking phase?
- What is the test that makes you say *yes*?
- Where does it **stop** — point at the step.

Keep the paper. You cannot write this as a program yet: it needs the machine to *choose* (Tuesday) and to *repeat* (week 3). Then you will.

Tuesday's algorithm, typed in

```
1 distance = input("Distance in km: ")
2 time      = input("Time in hours: ")
3
4 speed = distance / time
5
6 print("Average speed:", speed)
```

Does this work? **Which line?**

The machine is less forgiving than I was

```
TypeError: unsupported operand type(s) for /: 'str' and 'str'
```

- It tells you the **operator**: /
- It tells you the **types**: str and str
- It is not cryptic. It is a **diagnosis**.

Failure 1 — everything the user types is text

✗ Broken

```
distance = input("Distance: ")    # "100"  <- text
speed = distance / time            # TypeError
```

✓ Fixed

```
distance_km = float(input("Distance: "))
speed = distance_km / hours        # 50.0
```

`input()` returns `str`. Always. Even when every character is a digit.

The version that does not crash

```
a = input("First number: ")    # "2"  
b = input("Second number: ")  # "3"  
print(a + b)                  # 23    <- not 5
```

No error. No warning. **Wrong.**

From one measurement to an average

```
d1 = float(input("Distance 1 (km): "))
t1 = float(input("Time 1 (h): "))
d2 = float(input("Distance 2 (km): "))
t2 = float(input("Time 2 (h): "))
d3 = float(input("Distance 3 (km): "))
t3 = float(input("Time 3 (h): "))
v1 = d1 / t1
v2 = d2 / t2
v3 = d3 / t3
average = (v1 + v2 + v3) / 3
print("Average speed:", average, "km/h")
```

Test: (100, 2) (90, 1.5) (140, 2) $\rightarrow$ 50, 60, 70 $\rightarrow$ **60.0**

Thirty measurements this way? A moving average? *The program must repeat — week 3.*

Anatomy of a program

Part	What it does	Example
Comment	a note for humans	# calculate the speed
Multi-line comment	where your algorithm lives	"""INPUT: ... 1. ..."""
Variable	a named box holding a value	distance = 100
Assignment	puts a value in the box	speed = d / h
Expression	something with a value	d / h
Input	gets a value (as text)	input("Distance: ")
Output	shows a value	print(speed)
Function call	uses code someone else wrote	float(...), round(...)

READ → input() <- → = DISPLAY → print()

Every program: line 1 # your name, then the algorithm in """ """, then code.

Failure 2 — Python has its own opinion

1	()	parentheses
2	**	exponent — right to left
3	-x	negation
4	* / // %	left to right
5	+ -	left to right
6	< <= > >= == !=	comparison
7	not and or	logical

When you are unsure, parentheses cost nothing and buy certainty.

The one that will cost somebody marks

```
v1, v2, v3 = 50, 60, 70  
  
average = v1 + v2 + v3 / 3           # 133.33  
average = (v1 + v2 + v3) / 3        # 60.0
```

It runs. It prints a number. The number is nonsense.

Running is not a test.

- The average program ran. It was wrong.
- The `a + b` program ran. It was wrong.

Your only defence today: **know what you expect before you press run.**

Built-in functions — funciones predefinidas

```
abs(-7)           round(3.14159, 2)   max(3, 9, 2)
len("algoritmo")  int("42")                             float("2.5")

import math
math.sqrt(16)      math.pi              math.floor(3.9)
```

The distinction that matters

`round(3.7)` has a **value**: 4. You can store it.

`print(3.7)` has no value. It has an **effect**.

Exercise 1 — Predict, then run

Individual · 10 minutes

Open `examples/03_precedence.py`. **Do not run it.**

Write your prediction for all nine lines on paper. *Then* run it and mark each right or wrong.

Hand in the paper — **including the wrong ones**.
The wrong ones are the useful part. They are not penalised.

Exercise 2 — Trip cost

Plenary · built together · 20 minutes

Given distance (km), speed (km/h), fuel economy (km/litre) and price per litre:
display the travel time, the litres required, and the total cost in pesos.

1. **Algorithm on paper first.** I will look at it before anyone opens Thonny.
2. Code it from *your* algorithm.
3. Check: 200 km, 100 km/h, 12 km/l, \$24.50 $\rightarrow$ 2.0 h, 16.67 l, \$408.33

Exercise 3 — Celsius to Fahrenheit, in Trinket

Individual · 5 minutes here, finish at home

In **Trinket** — `trinket.strivemath.org/python3` — the quick-exam editor.

$$^{\circ}F = ^{\circ}C \times 9/5 + 32$$

Two required checks: $0 \rightarrow 32.0$, $100 \rightarrow 212.0$

If you wrote `c * 9 / 5 + 32` without thinking about precedence — were you **lucky** or **right**? Be able to say which.

1. **Homework 1** — BMI and seconds-to-time. Due **Sunday 27 Sep, 23:59**.
Handwritten algorithm is part of the submission.
2. Respondus practice exam — Monday 28 Sep, 23:59.
3. Levelling module: lessons 1–5 (as assigned). Whole module closes Thu 8 Oct.
4. Autoestudio: relational and logical expressions, decision structures.

Quick exam 1: Tuesday 29 September, first 20 minutes.

Algorithms · operator precedence · built-in functions · calculations

**A program that runs
is not a program that is correct.**